Frugalité des langages de programmation, liens avec les pratiques de développement

Éric Patrizio, Gabriel Alcaras, Matthieu Moy, Lionel Morel

Avril 2026

1 Contexte

L'industrie du logiciel fait face à une évolution très rapide et ininterrompue depuis les premiers jours de l'informatique, avec une apparition permanente de nouvelles technologies (traits de langages, frameworks, outils, montées de versions, processus, méthodes, etc.) permettant aux équipes de produire toujours plus vite. Le phénomène est accentué par des niveaux d'abstractions toujours plus élevés rendant le tout accessible au plus grand nombre, y compris des programmeurs ne maîtrisant pas certaines compétences fondamentales sous-jacentes. Alors que l'évolution des outils a permis cette démocratisation de l'activité de développement, cette situation engendre aussi souvent des projets souffrant d'une inflation non maîtrisée, techniquement endettés et difficilement maintenables.

Ce constat s'inscrit dans une crise écologique plus globale, largement documentée, décrite depuis le début des années 1970 [10] et qui pousse nos sociétés à s'adapter. Le champ du numérique n'est pas exempt d'évolution à cet égard. Plusieurs courants ont ainsi émergé ces dernières années, autour du "numérique responsable" ou du greenIT. Le travail proposé dans cette thèse s'inscrit dans une dynamique de sobriété numérique "qui consiste, dans le cadre d'une réflexion individuelle et collective, à questionner le besoin et l'usage des produits et services numériques dans un objectif d'équité et d'intérêt général. Cette démarche vise à concevoir, fabriquer, utiliser et traiter la fin de vie des équipements et services numériques en tenant compte des besoins sociaux fondamentaux et des limites planétaires. Pour cela il est nécessaire d'opérer des changements de politiques publiques, d'organisation, des modes de production et de consommation et plus globalement de mode de vie" [1]. L'équipe de recherche Phénix de l'INSA Lyon mène depuis quelques années des travaux autour des technologies logicielles qui permettent de penser la construction des systèmes numériques à l'intérieur des limites planétaires [9]. En particulier, elle s'intéresse à l'établissement de propriétés des langages de programmation pour rendre la construction des systèmes numériques réellement conviviale, selon la définition de Illich [8], c'est-à-dire permettant d'assurer notamment "l'autonomie créatrice" de l'utilisateur [13]. Phénix travaille en collaboration avec l'équipe CASH du LIP sur la question des langages de programmations.

S'intéresser aux pratiques de l'industrie du logiciel, c'est aussi, en amont, s'intéresser aux outils de production, à savoir les langages de programmation et les environnements de développements associés. Manifestement, le phénomène inflationniste est également de mise à ce niveau : toujours plus de traits de langage, de fonctionnalités, d'optimisations, d'analyses, de composants, de bibliothèques, etc. Le niveau de complexité des interprètes, machines virtuelles, compilateurs et outils de manipulation ne cesse de croître jusqu'à atteindre parfois des niveaux de technicité très avancés que seules quelques entreprises peuvent raisonnablement mener, et souvent loin d'assurer l'autonomie créatrice des programmeurs.

L'"*inflation logicielle*" est une expression générique qui consiste à caractériser la croissance continue d'un logiciel aussi bien en termes de consommation de ressources matérielles qu'en termes de complexité de sa conception. La complexité peut elle-même se caractériser par un certain nombre de métriques (nombre de lignes de code, nombre de modules, nombre de dépendances externes [3, 4], volume de données manipulées, par exemple) ainsi que des qualités plus subjectives (nombre de fonctionnalités peu ou non utilisées, dette technique, difficulté d'appropriation). Selon la loi de Wirth, cette inflation est clairement identifiée comme étant la cause de l'obsolescence matérielle [2]. Dans le cadre de la démarche de sobriété numérique mentionnée précédemment, nous définissons la **frugalité logicielle**, en opposition à l'inflation, comme l'ensemble des pratiques qui permettent de réduire l'ensemble des impacts des technologies numériques et de leurs usages, de tendre vers une simplification des objets, une réduction des dépendances, un ralentissement de l'évolution technique.

2 Sujet

Le projet de thèse vise à explorer la question de la frugalité des langages de programmation et de l'organisation du travail mis en œuvre pour les construire (conception, spécification, implémentation), les faire évoluer (organisation, animation de communautés d'utilisateurs, choix, gouvernance) et les utiliser (outillage, pratiques de développement logiciel). Nous traiterons des questions qui sont plus d'ordre sociologique, comme : Pourquoi un langage naît ? Qui initie son implémentation ? Quels sont les protocoles de sa maintenance évolutive ? Comment provoquer son adoption ? Quels contextes favorisent son inflation ? Mais également des questions d'ordre plus informatique : est-ce que la multiplication des concepts de programmation disponibles dans un langage tant à complexifier l'ensemble des outils utilisés pour manipuler les programmes ? Comment le rythme d'évolution des outils associés à un langage influence-t-il l'évolution des projets logiciels écrits avec ce langage ? La complexité croissante de son implémentation, d'un côté, et de sa prise en main (écosystème compris), de l'autre côté, sont-elles inéluctables ? L'inflation des langages encourage-t-elle l'inflation logicielle au sens plus large ?

Comme évoqué à l'instant, ces premières questions mettent en exergue deux aspects complémentaires : l'un technique, l'autre sociologique. Du point de vue technique, il s'agit d'identifier les caractéristiques des langages qui permettent de favoriser la frugalité sur l'ensemble de la chaîne de construction ou d'usage du langage ou à l'inverse tendent à la complexifier. Du point de vue sociologique, il s'agit d'identifier les pratiques organisationnelles qui influencent des choix techniques, voire des pratiques de développement.

Un paradigme de programmation est une manière d'appréhender un problème pour, in fine, le programmer. Parmi les paradigmes, les plus usuels sont l'impératif, l'orienté objet et le fonctionnel, et la définition et comparaison de ces différents modèles de programmation a déjà fait l'objet de travaux par le passé [14]. Chaque paradigme est associé à un modèle de pensée et il est habituel de les mélanger au sein d'un même langage. À très gros grain, l'approche impérative consiste à enchaîner, orchestrer, des séquences d'instructions. L'orienté objet consiste à regrouper, réutiliser, des données et des traitements. Le fonctionnel consiste à composer des fonctions. Côté utilisateurs, les programmeurs s'approprient dans le temps, avec la pratique, ces approches distinctes. Côté concepteurs, chaque paradigme impose une implémentation spécifique singulière des choix algorithmiques au sein des outils manipulant un langage donné (interprètes, compilateurs, typeurs, etc).

Dans cette thèse, nous avançons l'hypothèse suivante qui sera étudiée afin d'être validée ou invalidée : "*La multiplication des paradigmes de programmation au sein d'un même langage tend à favoriser l'inflation logicielle, aussi bien dans les outils de manipulation de ce langage que dans les logiciels construits avec celui-ci.*"

L'objectif principal de cette thèse est donc d'identifier et documenter les rouages qui, depuis la conceptualisation d'un langage jusqu'à son utilisation par des équipes de développement au sein de projets logiciels, favoriseraient, ou non, l'inflation et la complexification des outils permettant la manipulation du langage ou des logiciels construits avec ce langage et *a contrario* les rouages qui permettraient de tendre vers une frugalité, c'est-à-dire une limitation des ressources nécessaires à son usage et un ralentissement du renouvellement technique.

3 Exemple : cas des langages Lua et OCaml

D'un point de vue pratique, cette thèse consistera à comparer des langages aux caractéristiques suffisamment différentes. Les langages Lua [7, 6] et OCaml [12] pourraient être de bons candidats. En effet, tous deux nés à la même période (Lua en 1993, OCaml en 1996) dans le cadre de projets académiques (Lua à l'Université PUC-Rio au Brésil, OCaml à l'Université Paris-Diderot en France), ils ont suivi par la suite des trajectoires assez distinctes. Lua est un langage de script impératif dynamique, au-dessus duquel de multiples paradigmes (programmation orientée objet, fonctionnelle, modulaire) peuvent être proposés uniquement via une utilisation des tables associatives et des clôtures qui sont les seules constructions "avancées" proposées nativement par le langage. Son implémentation est restée compacte (environ 30 000 lignes de C) et a priori appropriable raisonnablement par des tiers. Le langage lui-même a très peu évolué en plus de 30 ans.

OCaml, déjà successeur de Caml né quelques années auparavant en 1985, implémentation des langages ML, est un langage impératif fonctionnel typé statiquement. OCaml n'a cessé de s'enrichir de multiples concepts suivant de près l'état de l'art de la recherche (types algébriques généralisés, ou plus récemment les effets algébriques, etc.) rendant de fait son implémentation accessible uniquement à une sélection d'experts. À titre d'exemple [5, 11], la taille de la distribution binaire a été multipliée par 10 entre la version 4.02.0 et la version 5.0.0 (à 10 ans de distance).

La thèse s'attachera à expliquer ce qui, aussi bien dans les choix de constructions de langages que dans les outils développés pour manipuler ces langages, a pu contribuer au succès de chacun pour tel ou tel domaine d'application. On identifiera les liens entre ces constructions de langage, le succès ou non d'un langage et sa frugalité.

4 Déroulement

La thèse débute le 01/06/2026 pour une durée de trois ans.

Durant cette période, le doctorant suivra approximativement les étapes suivantes :

— ≈ 6 mois : prise en main et exploration de plusieurs écosystèmes de programmation, par exemple Lua, exploration de l'écosystème de programmation. Montée en compétence sur les méthodologies d'ethnographie nécessaires pour les enquêtes socio-techniques
— ≈ 1 an : construction de grilles d'analyse des caractéristiques de langages de programmation. Mise en place et déploiement de métrique pour caractérisation quantitative des bases de code ;
— ≈ 6 mois : travail d'enquête auprès de communautés de programmation (Lua et OCaml) pour identification de critères de convivialité ;
— ≈ 6 mois : publication des résultats ;
— ≈ 6 mois : rédaction du manuscrit puis préparation de la soutenance.

5 Encadrement

La thèse sera hébergée dans l'équipe Phénix du CITI, avec Matthieu Moy comme directeur de thèse. Elle sera financée par le projet PEPR Numérique ÉcoResponsable, contractualisée par l'INSA de Lyon.

Matthieu Moy MCF HDR en informatique Lyon 1 au Laboratoire de l'Informatique du Parallélisme et membre externe de l'équipe Phénix du CITI, a travaillé sur des outils de simulation, de développement et de validation pour les systèmes embarqués. Depuis 2022, il redirige ses travaux de recherche vers la frugalité numérique. Il co-encadre en particulier une thèse CIFRE démarrée en 2025 avec la société Commown sur le développement d'outils low-tech pour l'infogérance.

Lionel Morel MCF en informatique INSA au laboratoire CITI, a travaillé sur la compilation et les modèles d'exécution de langage pour la programmation de systèmes embarqués critiques et l'évaluation de performance de programmes pour machines parallèles. Depuis 2021, au sein de l'équipe Phénix, il explore la question de la compatibilité des systèmes programmables avec les limites planétaires. À titre d'exemple, il a travaillé à l'identification de propriétés de convivialité de systèmes d'exploitation minimalistes.

Gabriel Alcaras Sociologue et chercheur postdoctoral à Science Po. Il a rejoint le médialab en septembre 2024. Il étudie l'impact des outils utilisés pour la construction des logiciels sur le travail.

Références

[1] Berthoud, Françoise ; Olivi, Martine ; Lefèvre, Laurent : Beaucoup de verrous, peu de leviers ! Sobriété numérique : le cas est grave mais pas désespéré. In : *JRES 2024 - Journées réseaux de l'enseignement et de la recherche*. Rennes, France, Dezember 2024

[2] Charles, Henri-Pierre ; Zeitoun, Xavier ; Berrée, Jean-François ; Peralta, Maxime ; Halter, Thibault : *Analyse de cycle de vie adapté au logiciel : analyses, outillages et recommandations*. Symposium pour l'électronique et le numérique durables. https://cea.hal.science/cea-05026005. Version : Dezember 2024. – Poster

[3] Cox, Joel ; Bouwers, Eric ; Eekelen, Marko van ; Visser, Joost : Measuring Dependency Freshness in Software Systems. In : *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering* Bd. 2, 2015, S. 109–118

[4] Cox, Russ : Surviving software dependencies. In : *Communications of the ACM* 62 (2019), Nr. 9, S. 36–43

[5] Fessant, Fabrice L. : *The Growth of the OCaml Distribution*. Blog article. https://ocamlpro.com/blog/2023_01_02_ocaml_distribution. Version : 2023. – Also discussed on the forum : https://discuss.ocaml.org/t/blog-the-growth-of-the-ocaml-binary-distribution/11202

[6] Ierusalimschy, Roberto ; de Figueiredo, Luiz H. ; Celes, Waldemar : The evolution of Lua, continued. In : *Journal of Computer Languages* 83 (2025), 101326. http://dx.doi.org/https://doi.org/10.1016/j.cola.2025.101326. – DOI https ://doi.org/10.1016/j.cola.2025.101326. – ISSN 2590–1184

[7] Ierusalimschy, Roberto ; Figueiredo, Luiz H. ; Celes, Waldemar : The evolution of Lua. In : *Proceedings of the third ACM SIGPLAN conference on History of programming languages*, 2007, S. 2–1

[8] Illich, Ivan : *La Convivialité*. 1973

[9] Lesueur, François ; Morel, Lionel ; Salagnac, Guillaume ; Stouls, Nicolas : *Proposition d'équipe phénix : Informatique frugale et sécurité émancipatrice pour un numérique de la transition*. https://phenix.citi-lab.fr/Docs/Propal.pdf, 2021

[10] Meadows, Donella H. ; Meadows, Dennis L. ; Randers, Jørgen ; Behrens, William W. : *The Limits to Growth*. Potomac Associates – Universe Books, 1972

[11] Mehnert, Hannes : *Requests for comments : how does opam-repository scale ?* Issue on GitHub. https://github.com/ocaml/opam-repository/issues/23789. Version : 2023

[12] Minsky, Yaron : OCaml for the masses. In : *Commun. ACM* 54 (2011), November, Nr. 11, 53–58. http://dx.doi.org/10.1145/2018396.2018413. – DOI 10.1145/2018396.2018413. – ISSN 0001–0782

[13] Morel, Lionel ; Salagnac, Guillaume ; Stouls, Nicolas : *Towards Self-Hosting Software for Small Computers*. https://lionel.morel.ouvaton.org/selfhosting.pdf, 2024

[14] Van Roy, Peter : Programming Paradigms for Dummies : What Every Programmer Should Know. (2012), 04